

# A universal LLM Framework for General Query Refinements

Eldar Hacohen  
Bar-Ilan University  
eldar.hacohen@live.biu.ac.il

Yuval Moskovitch  
Ben-Gurion University of the Negev  
yuvalmos@bgu.ac.il

Amit Somech  
Bar-Ilan University  
somecha@cs.biu.ac.il

## Abstract

Numerous studies have explored the SQL query refinement problem, where the objective is to minimally modify an input query so that it satisfies a specified set of constraints. However, these works typically target restricted classes of queries or constraints. We present OMNITUNE, a general framework for refining arbitrary SQL queries using LLM-based optimization by prompting (OPRO). OMNITUNE employs a two-step OPRO scheme that explores promising refinement subspaces and samples candidates within them, supported by concise history and skyline summaries for effective feedback. Experiments on a comprehensive benchmark demonstrate that OMNITUNE handles both previously studied refinement tasks and more complex scenarios beyond the scope of existing solutions.

## PVLDB Reference Format:

Eldar Hacohen, Yuval Moskovitch, and Amit Somech. A universal LLM Framework for General Query Refinements. PVLDB, 14(1): XXX-XXX, 2020. doi:XX.XX/XXX.XX

## PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://github.com/analysis-bots/omnitune>.

## 1 Introduction

The problem of SQL query refinement has been studied in previous work [17, 23, 37, 42, 46, 48, 49, 54], with the goal of minimally modifying a given query so that its results satisfy predefined constraints. Early work [46], for example, focuses on refining WHERE-clause predicates in conjunctive Select-Project-Join (SPJ) queries to meet cardinality constraints, adjusting queries that return too few or too many tuples. More recent studies [17, 42, 54] consider constraints defined over specific subgroups in the query output.

While effective for their specific target settings, existing approaches typically assume fixed notions of structured constraints and refinement, and are often restricted to relatively simple query classes. For example, [42] considers only SPJ queries with cardinality constraints, the work of [17] extends this setting to SPJ with an ORDER BY clause and considers cardinality constraints over subgroups within the top- $k$  results, and [54] consider range queries and are limited to the refinement of numerical predicates with constraint over a single binary attribute. In practice, however, users may wish to refine a much broader range of queries—including those with GROUP BY clauses or subqueries—and may impose application-specific definitions of both constraints and refinement criteria. To illustrate this gap, we next consider an example of a group-by query refinement problem.

*Example 1.1.* In Middle-earth, the Knightly Order plans to launch a pilot continental scholarship program aimed at identifying and supporting young, promising warriors. To this end, the Order tasks its chief analyst, Galadriel, with reviewing the pool of applicants

| id    | gender | region     | major      | GPA |
|-------|--------|------------|------------|-----|
| 1     | F      | Shire      | Archery    | 3.7 |
| 2     | M      | Rivendell  | Swordcraft | 3.8 |
| 3     | M      | Mordor     | Tactics    | 3.7 |
| 4     | F      | Rohan      | Melee      | 3.7 |
| 5     | M      | Erebor     | Scouting   | 3.8 |
| ...   | ...    | ...        | ...        | ... |
| 9996  | F      | Rohan      | Swordcraft | 3.8 |
| 9997  | F      | Khazad-dûm | Stealth    | 3.6 |
| 9998  | M      | Lothlórien | Tactics    | 3.9 |
| 9999  | F      | Shire      | Melee      | 3.6 |
| 10000 | M      | Rohan      | Archery    | 3.5 |

Table 1: The Middle-earth Students dataset  $D$ .

```
SELECT region, avg(GPA), COUNT_IF(gender = F), COUNT(*)
FROM MiddleEarth_Applicants
WHERE GPA > 3.5 AND major IN ('Tactics', 'Archery')
GROUP BY region
HAVING COUNT(*) > 100
```

Figure 1: Initial Scholarship Query  $Q$

and proposing initial selection criteria for the program’s first cycle. These criteria include: (i) a minimum GPA threshold, (ii) an approved set of majors, and (iii) a minimum representation quota per participating region.

Senior leadership further specifies two program-level constraints that must be satisfied by any proposed parameterization: (1) geographic coverage—at least eight distinct regions must be represented in the pilot cohort; and (2) gender balance—the overall ratio of female to male recipients should be approximately 1:1.

Galadriel examines the applications database (see Table 1 for a sample), consisting of five attributes: ID, gender, region, major, and grade point average (GPA). She issues an initial SQL query<sup>1</sup>, as depicted in Figure 1, representing an intuitively suitable criteria: GPA threshold of 3.5, majors are *Tactics* and *Archery*, and minimum region-quota of 100 students.

Galadriel now examines the results (see Table 2); unfortunately, none of the constraints are satisfied:

- (1) The initial scholarship design includes only 5 regions, fewer than the required 8.
- (2) The total number of female students is 170, out of 632 which is significantly below the required ratio of 0.5.

Consequently, Galadriel needs to *refine* the selection criteria in order to meet the stated constraints. □

Unfortunately, even the simple query of the example above is not supported by existing query refinement frameworks [17, 42, 46, 54], as none support the refinement of group-by queries with

<sup>1</sup>Note that the function *COUNT\_IF* is not supported in all databases, yet can be easily implemented in native SQL: `COUNT(CASE gender=F THEN 1 ELSE 0 END);`.

predicates in the HAVING clause, as well as free-form constraints. Addressing this would require to formally define the refinement problem, design a constraint satisfaction framework, and explore adaptations of existing solutions to support these query settings. Alternatively, refining the query manually can be a tedious and repetitive task. To this end, we present a universal framework for SQL query refinement problems. Given a database  $D$  and starter query  $Q$ , as well as notions for refinement distance and constraints (represented as Python functions), our framework iteratively seeks a refined query that is as close as possible to  $Q$  while (approximately) satisfying the constraints.

Our solution builds on the optimization-by-prompting (OPRO) paradigm [21, 62], where an LLM is iteratively prompted to propose candidate refinements based on a textual specification of the task, a description of the search domain, and the history of past candidates. However, as we show empirically, a direct application of this paradigm performs poorly, due to the vast combinatorial search space and the long, low-information interaction histories that fail to effectively guide subsequent proposals. We therefore design a dedicated two-step OPRO scheme for the query refinement problem. First, the LLM selects a promising refinement *subspace* that aggregates many potential refinements. Then, a subsequent LLM samples individual refinements that aim to satisfy the constraints while minimizing the distance from the original query  $Q$ .

We then design OMNITUNE, an efficient prototype system that instantiates our query-refinement OPRO scheme in practice. OMNITUNE addresses several core challenges, including representing an exponentially large space of refinement subspaces without materialization, constructing concise, query-dependent textual descriptions of the database and search domain, and providing informative yet compact guidance to LLM-based optimizers.

At a high level, OMNITUNE operates in two phases: (1) a lightweight pre-processing phase that derives tailored representations of the database and refinement domain, and (2) an optimization loop that alternates between selecting promising refinement subspaces and sampling concrete refinement assignments via two specialized LLM modules. Throughout optimization, OMNITUNE maintains a skyline of candidate assignments that captures the best observed trade-offs between query distance and constraint satisfaction; this skyline serves primarily as a multi-objective signal that guides both subspace selection and assignment generation, and additionally enables an effective early-stopping mechanism when the skyline stabilizes across iterations.

Our empirical evaluation demonstrates the effectiveness of our framework across a wide range of query refinement tasks: (i) Top- $k$  queries with group-representation constraints [17], (ii) range queries with group-ratio constraints [54], (iii) diversity-aware SPJ queries [42], and (iv) complex queries beyond the scope of prior dedicated approaches. We compare OMNITUNE against strong state-of-the-art LLMs, including OpenAI GPT-5-Thinking [50] and Google Gemini 2.5 Flash-Thinking [24]. Our results show that OMNITUNE achieves performance close to that of specialized refinement solutions on their native problem classes, while substantially outperforming LLM-based baselines and extending effective query refinement to previously unsupported settings.

| region     | Avg. GPA | Count_if(gender=F) | Count(*) |
|------------|----------|--------------------|----------|
| Rivendell  | 3.9      | 50                 | 150      |
| Gondor     | 3.7      | 33                 | 125      |
| Lothlórien | 3.8      | 32                 | 120      |
| Mordor     | 3.8      | 34                 | 135      |
| Erebor     | 3.7      | 21                 | 105      |

Table 2: Query output  $Q(D)$

We presented an early prototype of OMNITUNE [31], comprising two components: a wizard that translates natural-language specifications into formal query refinement problems, and an initial optimization engine. This work focuses on the latter, which we substantially extend and evaluate.

#### Contributions & Paper Outline.

- We formulate the UNIVERSAL QUERY REFINEMENT problem, a universal query refinement problem that provides a unified abstraction encompassing prior refinement settings and a wide range of new, previously unsupported scenarios (Section 2).
- We devise a dedicated LLM-based optimization-by-prompting (OPRO) scheme for query refinement, addressing the challenges of large, combinatorial refinement spaces via structured subspace selection and assignment sampling (Section 3).
- We design OMNITUNE, an end-to-end prototype instantiating our OPRO scheme, including query-conditioned database representations and skyline-based multi-objective (Section 4).
- We empirically evaluate OMNITUNE on a diverse collection of query refinement tasks adopted from prior work [17, 42, 54], as well as on more complex query patterns beyond the scope of existing solutions. The results show that OMNITUNE achieves performance close to specialized approaches while substantially outperforming strong LLM baselines (Section 5).

We survey related work in Section 6 and conclude in Section 7.

## 2 Preliminaries and Problem Definition

In this section, we provide an overview of the UNIVERSAL QUERY REFINEMENT problem. Notations are summarized in Table 3.

*Refinable predicates, supported queries.* We build on the notion of query refinement introduced in [46]. Given a query  $Q$  over a dataset  $D$ , a refinement of  $Q$  modifies one or more of its *refinable* predicates. We call a predicate in an SQL query *refinable* if it is of the form  $\langle A \text{ op } C \rangle$ , where  $A$  is an attribute in  $D$ ,  $\text{op}$  is a comparison operator, and  $C$  is a literal constant. The attribute  $A$  may be either a *base* (schema-native) attribute or a *derived* attribute, such as one obtained via aggregation or column-wise computation. If  $A$  is *numeric*, then  $\text{op} \in \{<, \leq, =, \geq, >\}$  and  $C \in \mathbb{R}$ . If  $A$  is *categorical*, then  $\text{op} = \text{IN}$  and  $C$  denotes a set of categorical values, such that  $C \subseteq \text{Dom}(A)$ , where  $\text{Dom}(A)$  is the set of distinct values appearing in column  $D(A)$ . We consider general SQL queries containing WHERE and/or HAVING clauses, with at least one refinable predicate. Let  $\text{Preds}(Q)$  denote the set of all *refinable* predicates in a query  $Q$ . For each predicate  $p \in \text{Preds}(Q)$ , we write  $p.A$ ,  $p.\text{op}$ , and  $p.C$  to denote its attribute, operator, and literal(s), respectively.

*Example 2.1.* Consider again the dataset  $D$  shown in Table 1 and the query  $Q$  in Figure 1. The set of refinable predicates in  $Q$

consists of three predicates. The first two appear in the WHERE clause:  $p_1 = \langle \text{GPA}, >, 3.5 \rangle$  and  $p_2 = \langle \text{major}, \text{IN}, \{\text{Tactics}, \text{Archery}\} \rangle$ . The third predicate appears in the HAVING clause and is given by  $p_3 = \langle \text{COUNT}(*), >, 100 \rangle$ .  $\square$

*Query refinement assignment.* Following [46], a valid refinement consists of modifying the literal(s) in  $p.C$  for each predicate  $p \in \text{Preds}(Q)$ . For predicates over numerical attributes, refinement is performed by directly altering the value in  $p.C$ , whereas for categorical predicates, refinement is achieved by adding to or removing items from the set specified by  $p.C$ .

A refinement assignment is denoted by  $\theta = \{\theta.p \mid p \in \text{Preds}(Q)\}$ , where each component  $\theta.p$  specifies a (possibly unchanged) refined literal  $p.C'$  for predicate  $p$ . We denote by  $Q_\theta$  the refined query obtained by applying assignment  $\theta$  to  $Q$ , and by  $\theta_0$  the *baseline* assignment, corresponding to the original query, that is,  $\theta_0.p = p.C$  for all  $p \in \text{Preds}(Q)$ .

*Example 2.2.* Consider again the database  $D$  and query  $Q$ , as depicted in Table 1 and Figure 1 (resp.). The baseline assignment  $\theta_0 = \{3.5, (\text{Tactics}, \text{Archery}), 100\}$ , corresponding with the original query  $Q$ . A new refinement assignment is thus  $\theta' = \{3.6, (\text{Tactics}), 80\}$ , which represents the following refined query  $Q_{\theta'}$  (modifications in **bold**):

```
SELECT region, AVG(GPA), COUNT_IF(gender = F), count(*)
FROM Students_Performance
WHERE GPA > 3.6 AND major IN ('Tactics')
GROUP BY region
HAVING COUNT(*) > 80
```

In the remainder of the paper, we use 'refinement' to denote both the assignment  $\theta$  and the resulting query  $Q_\theta$  interchangeably. We further denote the set of *all* possible assignments for  $Q$  by  $\mathcal{S}(Q)$ .

*Query output constraints.* Expanding the line of work on query refinement for output constraint satisfaction [17, 18, 42, 43, 54], we consider general constraints over the output. Since achieving perfect satisfaction may be infeasible [17], we adopt the notion of satisfaction deviation, relax the goal of exact satisfaction, and allow for approximate satisfaction. To this end, we define the output constraints as a function that measures the degree of violation of a query's result with respect to the constraints for a given database.

*Definition 2.3 (Constraint deviation function).* Given a refinement assignment  $\theta$ , a constraints deviation function  $\psi : \mathcal{S}(Q) \rightarrow [0, 1]$  depicts the degree of violation of the output of the refined query  $Q_\theta(D)$ . Given a tolerance  $\varepsilon \in [0, 1]$ , we say that an assignment  $\theta$  is  $\varepsilon$ -satisfying, if  $\psi(\theta) \leq \varepsilon$ .

By formulating constraints as functions, we enable the expression of any general constraint on the query output  $Q(D)$ .

*Example 2.4.* Recall the constraints specified in Example 2.1: (i) A 1:1 female-to-male ratio; and (ii) At least 8 distinct regions are represented. We first define a deviation function for the geographic coverage constraint. This function counts the number of regions returned by  $Q_\theta(D)$  and returns the normalized slack when fewer than 8 regions are selected, and 0 otherwise:

$$f^1(\theta) = \begin{cases} \frac{8 - |Q_\theta(D)|}{8}, & \text{if } |Q_\theta(D)| < 8, \\ 0, & \text{otherwise.} \end{cases}$$

The gender-balance deviation is measured by the (normalized) distance of the fraction of female students from the desired value  $1/2$ , aggregated across all regions:

$$f^2(\theta) = 2 \cdot \left| \frac{\text{SUM}(\# \text{Female}) \text{ in } Q_\theta(D)}{\text{SUM}(\text{Count}(*)) \text{ in } Q_\theta(D)} - \frac{1}{2} \right|.$$

Using  $f^1$  and  $f^2$ , we define the overall constraint deviation function  $\psi$  as their average:

$$\psi(\theta) = \frac{1}{2} (f^1(\theta) + f^2(\theta)).$$

Consider again the baseline assignment  $\theta_0$  corresponding to the original query  $Q$  (Figure 1) and its output  $Q(D)$ , shown in Table 2. The query result contains 5 distinct regions, and the ratio of female students among all selected students is 0.267. Therefore,

$$\psi(\theta_0) = \frac{1}{2} \left( \frac{8-5}{8} + 2 \cdot |0.267 - 0.5| \right) = \frac{1}{2} (0.375 + 0.466) = 0.421.$$

If  $\varepsilon$  is set to 0.1, then  $\theta_0$  is not  $\varepsilon$ -satisfying.  $\square$

In our solution, as described in Section 3, the constraint deviation function is provided to OMNITUNE as a Python code (see examples in our Github repository [29]).

*Refinement distance.* The query refinement problem aims to make *minimal* modifications to the query to satisfy the constraints. minimality may be defined in various ways, such as based on the set of output tuples of the queries [17, 54], known as outcome-based distance [17], or by the semantic distance in query predicates [17, 42], referred to as predicate-based distance [17]. It may also involve a combination of both approaches. To keep our framework general, we use a refinement distance function  $\Delta_{\theta_0, D} : \mathcal{S}(Q) \rightarrow \mathbb{R}_{\geq 0}$  that, given a refinement  $\theta \in \mathcal{S}(Q)$ , returns a value in  $\mathbb{R}_{\geq 0}$  that reflects the distance of  $\theta$  from  $\theta_0$  (possibly with respect to  $D$ , as in [54]). For brevity, when unambiguous, we write  $\Delta(\theta)$  in place of  $\Delta_{\theta_0, D}(\theta)$ .

*Example 2.5.* The work of [17] defined a predicate-based distance where the distance is measured with respect to each refinable predicate in the query. For a numerical predicate, the distance is defined as  $\frac{|p.C - p.C'|}{p.C}$ , where  $p.C'$  is the constant value of the predicate  $p$  in the refined query. The difference is normalized to show deviation from the original value of  $p$ . For categorical attributes, the distance is measured using Jaccard distance over the sets  $p.C$  and  $p.C'$ . The overall distance is then the sum of the per-predicate distance over all the refinable attributes. Consider again the refinement assignment  $\theta' = \{3.6, [\text{Tactics}], 80\}$  and the baseline assignment  $\theta_0 = \{3.5, [\text{Tactics}, \text{Archery}], 100\}$ , described in Example 2.2. Using a predicate-based  $\Delta(\theta')$  we obtain

$$\Delta(\theta') = \frac{|3.6 - 3.5|}{3.5} + \left(1 - \frac{1}{2}\right) + \frac{|100 - 80|}{100} = 0.029 + 0.5 + 0.2 = 0.729$$

We are now ready to formally define the UNIVERSAL QUERY REFINEMENT problem.

**Problem 1 (UNIVERSAL QUERY REFINEMENT).** Given a database  $D$ , a query  $Q$  with baseline assignment  $\theta_0$ , a constraint-deviation function  $\psi$  with tolerance threshold  $\varepsilon$ , and a query distance measure  $\Delta_{\theta_0, D}$ , we define an optimal refinement, denoted  $\theta^*$ , as:

$$\theta^* = \arg \min_{\theta \in \mathcal{S}(Q) : \psi(\theta) \leq \varepsilon} \Delta_{\theta_0, D}(\theta).$$

As discussed above, our formulation subsumes a wide range of prior work (e.g., [17, 42, 54]), each of which addresses a restricted variant of the problem (see Section 6), while also enabling many previously unsupported cases involving arbitrary constraints, refinement distance measures, and query types. We refer to a specific instantiation of the problem as a tuple

$$QRP = \langle Q, D, \psi, \epsilon, \Delta \rangle$$

We next present an OPRO scheme for solving any instance of the universal query refinement problem, followed by the OMNITUNE architecture that instantiates this approach. As shown in our experimental evaluation, our approach achieves performance close to problem-specific solutions, while naturally extending to a much broader class of instances covered by the universal formulation.

### 3 OPRO Scheme for Query Refinement

Our solution builds on the optimization-by-prompting (OPRO) paradigm [21, 62], a recent and promising approach for general black-box optimization. In OPRO, an LLM-based agent is given a natural-language specification of the objective and search domain, and iteratively proposes candidate solutions. Each selection is conditioned on the full interaction history, enabling the agent to adapt its decisions based on previously explored candidates and their evaluated outcomes.

We next describe a naïve, straightforward instantiation of the OPRO paradigm, followed by our dedicated, two-step OPRO scheme for the UNIVERSAL QUERY REFINEMENT problem.

*Naïve OPRO for Query Refinement.* Given an instance of the query refinement problem  $QRP = \langle Q, D, \psi, \epsilon, \Delta \rangle$ , the naïve approach repeatedly prompts the LLM to modify the query  $Q$  so as to produce an optimal refinement assignment  $\theta^*$ . For a fixed horizon of  $T$  iterations, at each iteration  $1 \leq t \leq T$ , the LLM is asked to generate a new candidate refinement assignment  $\theta_t$ , while being provided with the complete interaction history of prior attempts, including both distance and constraint-deviation scores, namely,

$$\{(\theta_{t'}, \Delta(\theta_{t'}), \psi(\theta_{t'}))\}_{t'=1}^{t' \leq t}.$$

However, as demonstrated in our experimental evaluation (Section 5), this naïve OPRO approach is largely ineffective. The underlying reasons are twofold: (i) the combinatorial size of the refinement search space, and (ii) the growing history of previous attempts, which, when presented in raw form, provides limited informative signal to guide subsequent refinements.

We therefore propose a two-step OPRO scheme. In the first step, the agent selects a promising refinement *subspace* that contains multiple candidate assignments. In the second step, the agent samples individual assignments from the selected subspace with the goal of identifying  $\epsilon$ -satisfying refinements that minimize the query distance from the original query  $Q$ . We next formally define refinement subspaces and then present our OPRO scheme.

*Refinement Subspace.* A refinement *subspace*, denoted  $\Theta$ , defines a *range* of assignments, by restricting the admissible values for each predicate in  $Preds(Q)$ . For predicates with numerical attributes, this restriction takes the form of an interval over the value domain,

| Notation                                  | Description                                                      |
|-------------------------------------------|------------------------------------------------------------------|
| $D; A; D(A)$                              | Database; Attribute; Column (att. $A$ projected on $D$ ).        |
| $Q; Q(D)$                                 | Query; Query results over $D$ .                                  |
| $Preds(Q)$                                | The set of refinable predicates in $Q$ .                         |
| $p = \langle p.A, p.op, p.C \rangle$      | Attribute, operator and constant of predicate $p$ .              |
| $Dom(A)$                                  | Set of all distinct values in column $D(A)$ .                    |
| $\theta; \theta_0$                        | A refinement assignment for $Q$ ; Baseline assignment.           |
| $\theta^*$                                | A minimal-distance $\epsilon$ -satisfying refinement assignment. |
| $S(Q)$                                    | The space of all individual assignments for query $Q$ .          |
| $\Theta; S^+(Q)$                          | Refinement subspace; Domain of all subspaces for $Q$ .           |
| $\psi, \epsilon$                          | Output constraints deviation function, tolerance thresh.         |
| $\Delta_{\theta_0}$ (or simply $\Delta$ ) | Refinement distance function.                                    |
| $T; K$                                    | Horizon (# of iterations); # of samples per iteration.           |
| $\mathcal{H}; \mathcal{H}_t$              | Refinement history; History at iteration $t$ .                   |
| $\mathcal{H}[\Theta_t]$                   | Local refinements history within subspace $\Theta_t$ .           |

Table 3: Notations Summary

while for categorical attributes it is defined using set inclusion, following the partial-order formulation of [53].

*Definition 3.1 (Refinement Subspace).* Given a query  $Q$  and database  $D$ , a refinement subspace is a collection  $\Theta = \{\Theta.p \mid p \in Preds(Q)\}$ , where each component  $\Theta.p$  constrains the admissible values for  $p.C$ . For numerical attributes,  $\Theta.p$  is an interval contained in  $[\min(Dom(p.A)), \max(Dom(p.A))]$ . For categorical attributes,  $\Theta.p$  is a set-inclusion range  $[C_{\min}, C_{\max}]$  with  $C_{\min} \subseteq C_{\max} \subseteq Dom(p.A)$ .

An assignment  $\theta$  is *compatible* with a subspace  $\Theta$ , if it assigns to each predicate a value that lies within the range permitted by that subspace. Namely,  $\theta \in \Theta \iff \theta.p \in \Theta.p \quad \forall p \in Preds(Q)$ .

*Example 3.2.* Recall again from our running example,  $Preds(Q)$  contains three predicates over the GPA, major, and COUNT(\*) attributes of query  $Q$  (see Figure 1). In Example 2.2 we examine two refinement assignments:  $\theta_0 = \{3.5, (Tactics, Archery), 100\}$ , and  $\theta' = \{3.6, (Tactics), 80\}$ .

Now, let us consider the following refinement *subspace*:

$$\Theta = \{[3.4, 3.7], [(Tactics), (Tactics, Melee, Stealth)], [70, 120]\}$$

See that  $\theta' \in \Theta$ , as  $3.5 \in [3.4, 3.7]$ ,  $(Tactics) \in [(Tactics), (Tactics, Melee, Stealth)]$ , and  $80 \in [70, 120]$ ; whereas  $\theta_0 \notin \Theta$  because  $(Tactics, Archery) \not\subseteq (Tactics, Melee, Stealth)$ .  $\square$

We denote the set of all possible refinements *subspaces* for query  $Q$  by  $S^+(Q)$ .

Building on the definition of refinement subspaces, we devise an OPRO scheme for the query refinement problem. Our two-step OPRO scheme is depicted in Algorithm 1. We next describe the inputs and algorithm steps:

**Inputs.** The system takes as input: (1) an instance of the UNIVERSAL QUERY REFINEMENT problem  $QRP = \langle Q, D, \psi, \epsilon, \Delta \rangle$ ; (2) the set  $S^+(Q)$ , representing the domain of all refinement subspaces (we describe below how a concise textual representation of  $S^+(Q)$  is constructed without explicitly materializing it); and (3) the horizon  $T$ , specifying the maximum number of iterations, and the number of assignments sampled from each subspace  $K$ , as detailed below.

**Initialization & History Management.** As mentioned above, in our iterative OPRO scheme, we keep the history of previously

---

**Algorithm 1:** Two-Step OPRO Scheme for Query Refinement

---

**Input:** Query refinement instance  $QRP = \langle Q, D, \psi, \varepsilon, \Delta \rangle$ ;  
Refinement subspaces domain  $S^+(Q)$ ; horizon  $T$ ; samples  
per iteration  $K$   
**Output:**  $\varepsilon$ -satisfying assignment  $\theta$  with min. refinement distance

```
1  $\mathcal{H} \leftarrow \emptyset$  // Initialize refinement history
2  $SAT \leftarrow \emptyset$  // Initialize constraint-satisfying set
3 for  $t = 1$  to  $T$  do
4    $\Theta_t \leftarrow \mathcal{M}^s(S(Q), \mathcal{H})$  // Suggest subspace
5   if  $\Theta_t \notin \mathcal{H}$  then
6      $\mathcal{H}[\Theta_t] \leftarrow \emptyset$  // Initialize subspace-specific history
7     for  $k = 1$  to  $K$  do
8        $\theta_k \leftarrow \mathcal{M}^a(\Theta_t, \mathcal{H}[\Theta_t])$  // Suggest refinement
9       for  $\Theta \in \mathcal{H}$  do // Update history
10        if  $\theta_k \in \Theta$  then
11           $\mathcal{H}[\Theta] \leftarrow \mathcal{H}[\Theta] \cup \{(\theta_k, \Delta(\theta_k), \psi(\theta_k))\}$ 
12        if  $\psi(\theta_k) \leq \varepsilon$  then // If constraints are met
13           $SAT \leftarrow SAT \cup \{\theta_k\}$ 
14 return  $\arg \min_{\theta \in SAT} \Delta(\theta)$ 
```

---

chosen assignments. We use a key-value structure, denoted  $\mathcal{H}$  (Line 1), in which the keys are unique subspaces  $\Theta_0, \Theta_1, \dots$ ; and the values are lists of all individual assignments examined thus far, that are compatible with each  $\Theta \in \mathcal{H}$ , along with their distance and satisfaction values. Namely,  $\mathcal{H}[\Theta] = \{(\theta, \Delta(\theta), \psi(\theta)) \mid \theta \in \Theta\}$ . We also initialize the set  $SAT$  (Line 2), which will contain all  $\varepsilon$ -satisfying assignments, throughout the optimization process.

**Subspace-Selection Step (LLM Call #1).** At each iteration  $t$ , we abstractly model the first step as an LLM call that selects the next *refinement subspace*  $\Theta_t \in S^+(Q)$ . This call, denoted  $\mathcal{M}^s$ , takes as input the optimization history  $\mathcal{H}$  (Line 4) and outputs a candidate subspace  $\Theta_t$ . In Section 4, we detail how this abstract call is instantiated in OMNITUNE by the *SubspaceLM* module, which provides the LLM with a concise, textual representation for  $S^+(Q)$  and  $\mathcal{H}$ .

**Assignment-Sampling Step (LLM Call #2).** Given the selected subspace  $\Theta_t$ , we issue  $K$  consecutive LLM calls that sample *individual* refinement assignments  $\{\theta_k \in \Theta_t\}_{k=1}^K$ . We denote this abstract LLM call by  $\mathcal{M}^a$  (Line 8), where each call is conditioned on the subspace-specific history  $\mathcal{H}[\Theta_t]$ , which contains assignments previously sampled from  $\Theta_t$ . In OMNITUNE, this call is instantiated by the *AssignmentLM* module (Section 4). After each call, the global history  $\mathcal{H}$  is updated, such that each previously generated subspace  $\Theta \in \mathcal{H}$  is appended with the currently sampled assignment  $\theta_k$  whenever  $\theta_k \in \Theta$  (Lines 9–11). Any  $\theta_k$  that is  $\varepsilon$ -satisfying is appended to  $SAT$  (Lines 12–13).

**Termination & Return value.** Finally, after  $T$  iterations, we return the set of  $\varepsilon$ -satisfying assignments stored in  $SAT$  that attain the minimum query distance from the baseline assignment  $\theta_0$  (Line 14).

We next describe the architecture of OMNITUNE, designed in accordance with Algorithm 1. The architecture incorporates several key components, including concise yet effective textual representations of  $D$  and  $S^+(Q)$ , mechanisms for summarizing subspace-history information  $\mathcal{H}[\Theta_t]$ , skyline-based objective definitions for realistic optimization, and an early-stopping mechanism.

## 4 OMNITUNE Architecture

We designed OMNITUNE, an efficient system that implements the two-step OPRO scheme for query refinement, as described above. OMNITUNE overcomes numerous challenges, such as producing concise textual representations for the database  $D$ , subspace domain  $S^+(Q)$ , and history  $\mathcal{H}$ , as well as introducing realistic objectives for the LLM calls, and early stopping mechanisms.

**Workflow Overview.** Figure 2 illustrates the overall architecture and execution flow of OMNITUNE. Given a query refinement problem instance  $QRP = \langle Q, D, \psi, \varepsilon, \Delta \rangle$ , OMNITUNE operates in two main phases. In the *pre-processing* phase, OMNITUNE constructs concise yet informative textual representations of the database  $D$ , tailored to the input query  $Q$ , as well as a representation of the refinement subspace domain  $S^+(Q)$ , which serves as the search space for subspace generation. In the *optimization loop* phase, OMNITUNE iteratively issues LLM calls via two specialized modules, in accordance with Algorithm 1: The *SubspaceLM* module generates a candidate refinement subspace  $\Theta_t$ , while the *AssignmentLM* module samples individual refinement assignments from  $\Theta_t$ . To guide this process, OMNITUNE maintains a compact summary of the interaction history  $\mathcal{H}$ , which provides the *SubspaceLM* with high-level statistics over previously explored subspaces  $\{\Theta_{t'} \mid t' < t\}$ . In addition, both modules are guided by skyline-based objectives derived from the set of previously observed assignments. Intuitively, the skyline maintains a set of high-quality assignments that represent the best trade-offs observed so far between query distance and constraint satisfaction.

Finally, OMNITUNE employs a skyline-based early-stopping mechanism that directly builds on this trade-off view: when the skyline remains unchanged for several consecutive iterations, indicating that no new assignment improves the current trade-offs between distance and satisfaction, the system halts and returns the best assignment found so far. We next elaborate on each part of the system. We start with the preprocessing phase and then describe in details the optimization loop modules.

### 4.1 Pre-processing phase

As mentioned above, in this phase, given the problem instance  $QRP = \langle Q, D, \psi, \varepsilon, \Delta \rangle$ , we first generate a concise description of the database  $D$  and refinement subspace domain  $S^+(Q)$ :

**1. Q-driven Database description.** Producing an effective and concise database schema description is a well-recognized challenge in LLM-based data management tasks, including text-to-SQL [35, 59], LLM-based schema matching [44], and LLM-driven BI and analytics [33]. Overly long or exhaustive descriptions often degrade LLM performance (see, e.g., the context rot problem [40]), motivating the need for concise and focused task descriptions.

To that end, rather than providing an exhaustive description of the entire database  $D$ , we derive a concise, query-conditioned representation tailored to the given query refinement instance (see Figure 4a for an example). This representation includes only the tables and attributes that appear in the query  $Q$ . Specifically, for each table referenced in  $Q$ , we describe its primary and foreign keys, as well as the names and data types of the attributes used in  $Q$ . In addition, we provide summary statistics for each such attribute: for

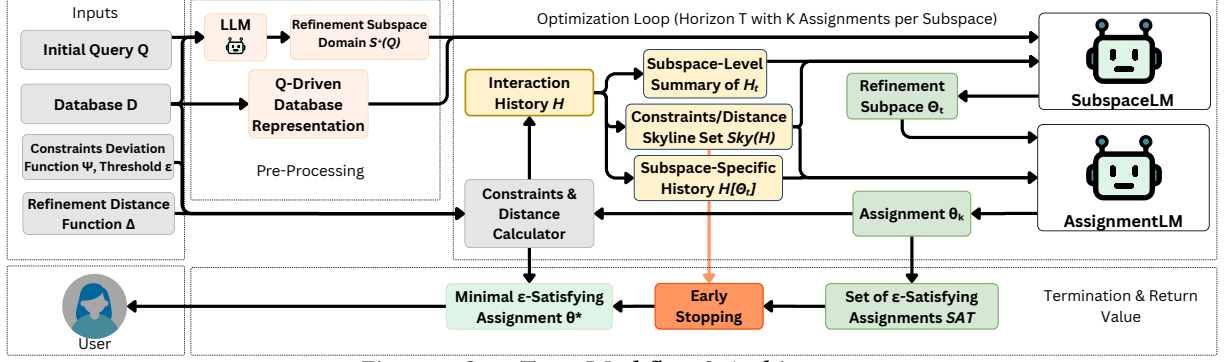


Figure 2: OMNITUNE Workflow & Architecture

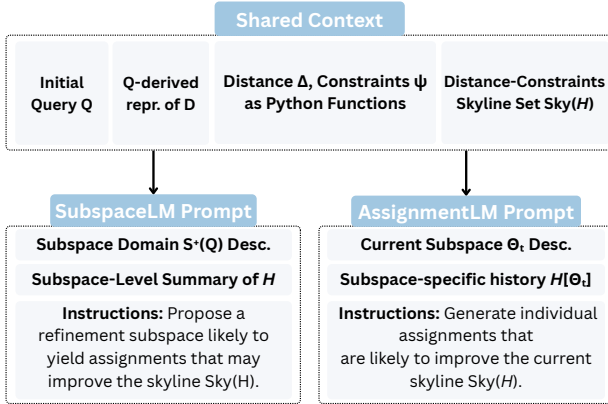


Figure 3: Prompts Formulation

numeric columns, the mean, standard deviation, and quartile values; and for categorical columns, the prevalence of each category.

**2. Description of the Subspace Domain  $S^+(Q)$ .** The goal in this component is to provide the *SubspaceLM* module with a compact yet expressive description of the refinement subspace domain  $S^+(Q)$ , from which refinement subspaces  $\Theta_t$  can be generated. Explicitly materializing  $S^+(Q)$  is infeasible, as it contains all possible combinations of per-predicate value ranges and would far exceed the context limitations of current LLMs.

Instead, we construct a concise characterization of  $S^+(Q)$  by specifying, for each refinable predicate  $p \in \text{Preds}(Q)$ , the maximal admissible range of values for  $p.C$ . For *base attributes* appearing directly in the database  $D$ , this process is straightforward: for numerical attributes, we compute the minimum and maximum values observed in the data, and for categorical attributes, we provide the set of distinct values. However, handling *derived* or *computed* attributes—such as those appearing in a HAVING clause—poses a greater challenge.

To address this case, we employ an LLM-assisted approach in which the model is prompted to generate auxiliary SQL queries that compute the minimum and maximum values of each derived field. For example, in our running scenario (Example 2.2), where the HAVING clause includes the refinable predicate  $\text{COUNT}(\ast) > 100$ , the LLM produces the following query:

```
SELECT MIN(size), MAX(size)
FROM (SELECT COUNT(*) AS size
      FROM MiddleEarth_Applicants
      GROUP BY region)
```

An example representation of  $S^+(Q)$  is depicted in Figure 4b. We provide the exact prompt used in our implementation in [29].

## 4.2 Optimization loop, prompt formulations

In accordance with Algorithm 1, the main optimization loop runs for a horizon of  $T$  iterations. At each iteration  $t$ , OMNITUNE invokes the *SubspaceLM* module to generate a refinement subspace  $\Theta_t$ , and then issues  $K$  calls to the *AssignmentLM* module to sample individual assignments from  $\Theta_t$ , while maintaining the interaction history  $\mathcal{H}$ . We next describe the *SubspaceLM* and *AssignmentLM* modules, with a focus on the formulation of their respective prompts (see Figure 3 for illustration). In particular, we introduce two techniques that substantially enhance optimization performance, as demonstrated in our experimental evaluation (Section 5): (i) a skyline-based objective, used in place of naïve instructions to generate the “best” subspace or assignment; and (ii) a concise, subspace-level summary of the history  $\mathcal{H}$ , employed by the *SubspaceLM* module. In [29], we provide the exact prompt templates, as well as numerous example workflows of OMNITUNE.

*Shared information.* The following items are included in both the *SubspaceLM* and *AssignmentLM* prompts:

**1. Description of the instance.** We begin with a brief, task-agnostic overview of the query refinement problem, then describe the input instance  $\langle Q, D, \psi, \epsilon, \Delta \rangle$  as follows: The  $Q$ -derived representation of the database  $D$ , as described above, together with the query  $Q$  itself. In addition, the distance function  $\Delta$  and the constraint-deviation function  $\psi$  are provided as executable Python code. Code-based representations are widely regarded as particularly effective for LLM-based reasoning and instruction following [20].

**2. Skyline-based Objectives.** Prior work has shown that clear, context-rich instructions can substantially improve zero-shot LLM performance across domains (e.g., [55, 56]). However, in our setting, naively prompting an LLM to generate an “optimal” refinement subspace  $\Theta_t$  or assignment  $\theta_k$  proves ineffective (see Section 5).

Existing OPRO solutions [62] typically instruct the LLM, at each iteration, to surpass the “best” solution obtained so far with respect to a single objective. In contrast, our setting is inherently multi-objective, as assignments must be  $\epsilon$ -satisfying w.r.t.  $\psi$  while simultaneously minimizing the distance  $\Delta$ . To this end, we devise a skyline-based objective.

Intuitively, a *skyline* [14] captures the set of “best” assignments by retaining only those for which no other assignment achieves both a smaller query distance  $\Delta(\theta)$  and a strictly better constraint deviation  $\psi(\theta)$ . In our setting, all  $\varepsilon$ -satisfying assignments are treated as equivalent with respect to constraint satisfaction, which we capture via the *constraints-threshold* deviation function

$$\psi_\varepsilon(\theta) \triangleq \min\{\psi(\theta), \varepsilon\}$$

An assignment  $\theta \in \mathcal{H}$  is said to belong to the skyline set, denoted  $\theta \in \text{Sky}(\mathcal{H})$ , if and only if it is not dominated by any other assignment  $\theta' \in \mathcal{H}$ , that is,

$$\theta \in \text{Sky}(\mathcal{H}) \Leftrightarrow \nexists \theta' \in \mathcal{H} : \left( (\Delta(\theta') < \Delta(\theta) \wedge \psi_\varepsilon(\theta') \leq \psi_\varepsilon(\theta)) \vee (\psi_\varepsilon(\theta') < \psi_\varepsilon(\theta) \wedge \Delta(\theta') \leq \Delta(\theta)) \right)$$

An example skyline set is depicted in Figure 4c. We leverage the skyline set  $\text{Sky}(\mathcal{H})$  when instructing the LLM in both stages of our procedure (see Figure 3): During subspace generation, the LLM is instructed to *propose a refinement subspace that is likely to include assignments capable of entering the skyline*. During assignment sampling, the LLM is instructed to *generate an individual refinement assignment that could join the current skyline set*

*The SubspaceLM prompt (LLM Call #1).* Given the problem instance description and the current skyline, as introduced above, the LLM is instructed to propose a refinement subspace likely to yield assignments that may improve the skyline. Rather than providing the full interaction history  $\mathcal{H}$ , we supply a concise yet informative *subspace-level* summary (see Figure 4d for an example). Specifically, for each previously explored subspace  $\Theta_{t'}$  with  $t' < t$ , we present aggregate statistics over the assignments sampled from that subspace, as recorded in  $\mathcal{H}[\Theta_{t'}]$ , including the total number of sampled assignments  $|\mathcal{H}[\Theta_{t'}]|$ , as well as the median and standard deviation of their distance and constraint-deviation scores.

*AssignmentLM Prompt (LLM Call #2).* Given the refinement subspace  $\Theta_t$  selected by the *SubspaceLM* module, the *AssignmentLM* is invoked  $K$  times to generate individual assignments  $\theta_k$  that are likely to improve the current skyline  $\text{Sky}(\mathcal{H}_t)$  where  $\mathcal{H}_t$  is the history  $\mathcal{H}$  at iteration  $t$ . As in the subspace-generation step, providing the full history  $\mathcal{H}$  is unnecessary; instead, the prompt includes only the subspace-specific history  $\mathcal{H}[\Theta_t]$ , containing assignments previously sampled from the current subspace.

*Early stopping criteria.* Finally, we introduce an early-stopping mechanism[52] based on the skyline set  $\text{Sky}(\mathcal{H})$  defined above, which allows OMNITUNE to terminate the optimization process before reaching the full horizon  $T$ , thereby reducing LLM token consumption. At iteration  $t$ , OMNITUNE halts if:

$$\text{Sky}(\mathcal{H}_t) = \text{Sky}(\mathcal{H}_{t-u})$$

That is, if the skyline set remains unchanged for  $u$  consecutive iterations. Intuitively, this condition indicates that the system has failed to identify a new assignment that dominates previously observed ones with respect to both refinement distance  $\Delta$  and constraint satisfaction  $\psi$ . In such cases, OMNITUNE terminates and returns the  $\varepsilon$ -satisfying assignment in SAT with minimum distance from the baseline assignment (see Line 14 in Algorithm 1). In our implementation, we set  $u = 2$ .

## 5 Experimental Setup and Evaluation

We performed an experimental analysis of our proposed solution using a dedicated benchmark composed of real-life datasets and considering realistic scenarios utilizing the queries and constraints used in prior works. We first examine the ability of OMNITUNE to successfully produce refinements as well as the quality of the generated refinements, comparing the results across different LLM-based baselines. We conduct an ablation study to evaluate the impact of our two-step OPRO scheme and the history representation component used in our system implementation on the resulting refinements. We conclude with an assessment of the effect of the horizon ( $T$ ) and the number of refinements per subspace ( $K$ ) on the quality of the resulting refinement and the token costs of generating it.

### 5.1 Experimental Setup

*Implementation.* OMNITUNE is implemented in Python 3 and uses DuckDB[26]. The structural prompt verification utilizes the Pydantic Python implementation [13]. The SubspaceLM and AssignmentLM components are built into LangChain [11] and support various LLM provider APIs. Across all experiments, the decoding parameters were fixed to temperature = 0 and top- $p$  = 0. All experiments were run on a 24-core CPU server. The code and data are provided in [29]. As default setting, we set the horizon  $T$  and the number of refinements per iteration  $K$  to 5, early stopping criterion  $u = 2$ , and use GPT-4o-mini.

*Benchmark.* To evaluate OMNITUNE, we constructed a unified benchmark for UNIVERSAL QUERY REFINEMENT instances. Each instance comprises a dataset, a query, a constraint function along with a tolerance value, and a distance function. The benchmark encompasses 32 instances utilizing 11 real-world datasets of varying sizes and attribute dimensions, including adaptations of instances used in prior works [17, 42, 54]. In particular, we consider 4 instance classes: **(1) Top- $k$**  queries as presented in [17] with constraint representation over the representation of different groups in the top- $k$  results, **(2) Range** [54] queries with constraint over the ratio of data groups in the output, **(3) Diversity** in the output of SPJ queries as was studied in [42], and **(4) Complex** queries that are not supported by previous works. For each instance class (1)-(3), we used the queries, constraints, and distance functions as defined in the corresponding prior work. In addition, we introduced four new instances for the Range class and two additional instances to the Diversity class, thereby enriching the benchmark and increasing the total number of instances per class to eight. A summary of the instances, including their class, properties of the queries, number of constraints, deviation values, and datasets used, is depicted in Table 5, and their full description is available in [29].

*Evaluating Metrics.* We evaluated the performance of the proposed solution using two criteria: *success rate* and *optimality*. The *success rate* measures the number of tasks we were able to obtain a refinement that  $\varepsilon$ -satisfies the constraints function. For each successful execution, we measure the *optimality* of the resulting refinement  $\theta$  with respect to the resulting refinement  $\theta_{opt}$  produced by a task-specific dedicated solution for the instance class (i.e., [17] for top- $k$  instances, [42] for diversity and [54] for range).

```
{ "table": "MiddleEarth_Applicants", "primary_key": "id",
  "attributes": {
    "GPA": { "type": "float", "domain": [2.0, 4.0],
      "stats": { "mean": 3.12, "std": 0.41,
        "q1": 2.85, "median": 3.10, "q3": 3.40 },
      ...
    "gender": { "type": "category", "domain": ["F", "M"],
      "stats": { "distribution": [ { "value": "F", "p": 0.46 },
        { "value": "M", "p": 0.54 } ] } } } }
```

(a)  $Q$ -driven Database Schema Description.

```
[{ "assignment_id": 1, "vals": { 3.7, "(Tactics, Archery)", 90 },
  "dev": 0.49, "dist": 0.20 },
{ "assignment_id": 7, "vals": { 3.6, "(Tactics, Archery)", 80 },
  "dev": 0.24, "dist": 0.23 },
...
{ "assignment_id": 13, "vals": { 3.9, "(Tactics, Melee)", 100 },
  "dev": 0.20, "dist": 0.57 } ]
```

(c) Skyline Set  $\text{Sky}(\mathcal{H})$ .

```
[{ "attribute": "gpa",
  "value_range": { "min_val": 2, "max_val": 4 } },
{ "attribute": "major",
  "value_range": { "min_val": [], "max_val":
    [Archery, Melee, Spearmanship, Tactics,
    Camouflage, Scouting, Swordship, Horsemanship] } },
{ "attribute": "COUNT(*)",
  "value_range": { "min_val": 34, "max_val": 452 } } ]
```

(b) Subspaces Domain Definition  $S^+(Q)$

```
[{ "subspace_id": 1,
  "subspace": { "gpa": [3.4, 3.8],
    "major": ["Tactics", "Archery"],
    "Count(*)": [60, 90] },
  "history_size": 5,
  "stats": { "deviation": { "med": .49, "std": .25 },
    "distance": { "med": .20, "std": .03 } } },
{ "subspace_id": 2, ... } ]
```

(d) Subspace-level History Summary.

Figure 4: Examples of textual representations of OMNITUNE objects

| Class          | Databases Used                                                              |
|----------------|-----------------------------------------------------------------------------|
| Top- $k$ [17]  | Astronauts [2], Law Students [9], MEPS [1], TPC-H [6]                       |
| Range [54]     | Texas Tribune [10], COMPAS [3], Housing Prices [8],<br>Fraud Detection [12] |
| Diversity [42] | Healthcare [7], ACSIncome [4], COMPAS, Students [5]                         |
| Complex (new)  | Law Students, Texas Tribune, TPC-H                                          |

Figure 5: Datasets used per instance class

The optimality metric is defined as

$$\text{Opt}(\theta) = \frac{\Delta_{\max} - \Delta(\theta)}{\Delta_{\max} - \Delta(\theta_{\text{opt}})}$$

where  $\Delta_{\max}$  is the max possible distance value for the given  $\Delta$  (e.g., 1 for Jaccard-based distance function, or the  $|Preds(Q)|$  for predicate-based distances) and is used to normalize the result. Since there is no dedicated solution that can handle instances in the complex class, we set  $\Delta(\theta_{\text{opt}})$  to 0 for these instances, which provides a lower bound on the possible distance value. Each experiment was repeated 5 times, and we report the success rate and the maximal Opt score obtained for each task.

**Baselines.** Our implementation of OMNITUNE interacts with LLMs via a standard API, making it compatible with any general-purpose model. To evaluate the performance of OMNITUNE, we used three lightweight, general-purpose LLMs:

- **ChatGPT Base** OpenAI GPT-4.1-mini [51]
- **Gemini Base** Google Gemini 2.0 Flash-Lite [24]
- **Mistral Base** Mistral 3.1 Small 24B-Instruct [47]

To the best of our knowledge, there is no existing solution that can handle the UNIVERSAL QUERY REFINEMENT problem. As our framework utilizes LLMs, they serve as a natural baseline for performance comparison. To this end, we considered the above lightweight, general-purpose LLMs, which we used as part of OMNITUNE, as standalone baseline solutions. Additionally, as our framework performs iterative multi-step computation, we further considered models that implement explicit chain-of-thought or multi-stage reasoning pipelines internally. To this end, we used

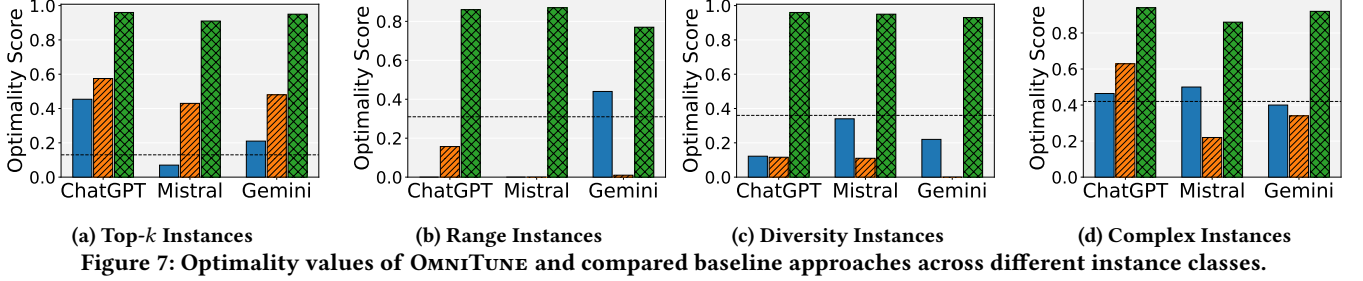
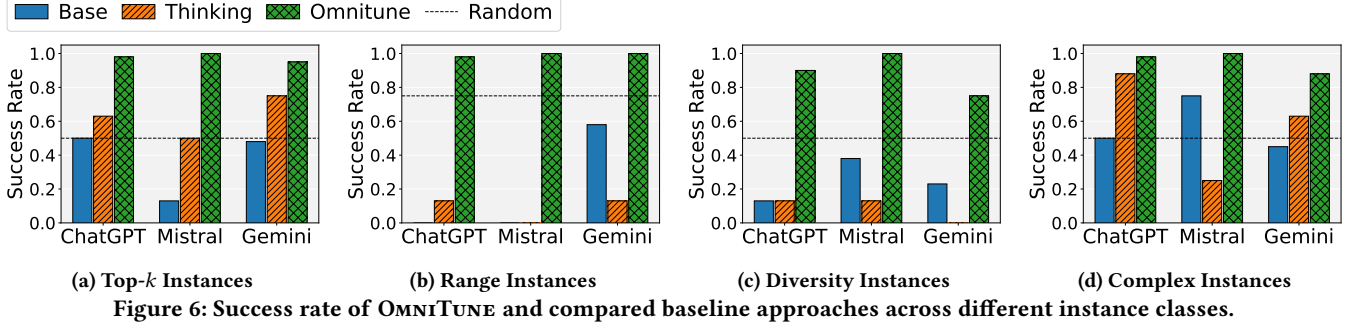
- **ChatGPT Thinking** OpenAI GPT-5-Thinking [50]
- **Gemini Thinking** Google Gemini 2.5 Flash-Thinking [24]
- **Mistral Thinking** Mistral Magistral Small-Thinking [47]

Both the Base and Thinking LLMs, were presented with  $QRP$  (using the same representation as in OMNITUNE), and asked to refine the query  $Q$ . At each iteration, we also provided the history of previous outputs. The models were stopped after  $T \cdot K$  iterations (25), yet all terminated earlier, due to the limit of their context. See [29] for the full implementation details of these baselines. In addition to the LLM-based baselines, we implemented a **sampling-based baseline**, which is given the search-space ranges for each of the refineable predicates and randomly samples 100 assignments uniformly from the given ranges. The  $\epsilon$ -satisfying assignment (if such a refinement exists) with  $\theta$  with minimal  $\Delta$  is returned.

#### Results Summary

- OMNITUNE is able to handle a wide range of instance classes of UNIVERSAL QUERY REFINEMENT, including instances with existing dedicated solutions and instances that are not supported by existing solutions, and outperform all baselines.
- OMNITUNE performs consistently across different LLMs, showing that the approach is robust and model-agnostic.
- The 2-steps OPRO scheme for query refinement employed by OMNITUNE plays a central role in achieving high performance as opposed to a naive OPRO implementation.
- The OMNITUNE components for summarized history representation, together with the skyline component, significantly improve performance.
- While there is a natural tradeoff between the quality metrics and token cost when varying the horizon  $T$  and the refinement per subspace  $K$ , the configuration of  $T = 5$  and  $K = 5$  yields an empirical optimal tradeoff.





## 5.2 Success rates and optimality

The first set of experiments aims to evaluate the success rates and optimality scores of our solution in comparison to baseline approaches. Figures 6 and 7 depict the success rate and optimality, respectively, for all instances and baselines. The lightweight, general-purpose LLMs are denoted as Base (blue bars), multi-stage reasoning models are denoted as Thinking (orange bar), and the performance of OMNITUNE is shown in green. Each category on the x-axis represents a model family, e.g., in the ChatGPT cluster group in Figure 6a, the success rate of OpenAI GPT-4.1-mini is shown in blue, OpenAI GPT-5-Thinking in orange, and the success rate of OMNITUNE using GPT-4.1-mini is shown in green. The success rate of the sampling-based baseline solution is shown as a dashed line.

As shown in Figure 6, OMNITUNE outperforms all baselines in all settings, achieving a success rate of 75%-100%, while the competitors' success rate was much lower, below 50% in most cases. In Particular, for Range and Diversity instances, LLM-based competitors perform worse than the random sampling solution, where, in some cases, they completely failed to generate  $\epsilon$ -satisfying assignments (as shown in Figures 6b and 6c, respectively). Moreover, the success rate of OMNITUNE remained consistent across different models, illustrating the robustness of our solution.

The optimality of the resulting refinements is shown in Figure 7. Similarly to the success rate, OMNITUNE outperforms all other competitors. The observed optimality scores for refinements generated by OMNITUNE range from 77% to 100%, with consistent performance across different LLM models. In contrast, the maximal optimality score for a refinement generated by a competitor was 62% (OpenAI GPT-5-Thinking over complex instance shown in Figure 7-complex) and an average optimality of 37% overall.

## 5.3 Ablation Study

The second set of experiments evaluates the usefulness of components in the OPRO scheme as well as the OMNITUNE system.

*Ablation for the two-step OPRO scheme.* To evaluate the effectiveness of the two-step OPRO scheme, we compared the performance of OMNITUNE to two variants of the system:

- (1) *No-SubspaceLM*, which omits subspace selection, forcing the AssignmentLM to explore the full refinement space  $\mathcal{S}(Q)$ . This corresponds to a naïve instantiation of the OPRO paradigm.
- (2) *No-AssignmentLM*. In this variant of the system, the SubspaceLM agent generates subspaces, and the AssignmentLM is replaced with a simple random sampling process that selects five random refinements from the subspace.

The success rate and the optimality values of the generated refinements are presented in Table 4. The results highlight the need for both AssignmentLM and SubspaceLM. In particular, we observed a substantial drop in performance for No-SubspaceLM, with up to a 62.5% decrease in success rate and a 63.7% decrease in optimality values for the Diversity instances when using the Mistral model, reflecting the difficulty of unguided exploration in large refinement spaces. While the No-AssignmentLM performs better than No-SubspaceLM in most cases, it still achieves lower success scores than the full system, underscoring the necessity of AssignmentLM.

*Ablation for OMNITUNE history component implementation.* The second ablation study focused on the effect of the history representation component presented in Section 4 on the preference of OMNITUNE. In particular, we aim to study the advantage of using the skyline component of OMNITUNE. We therefore considered the following alternative representation of the history:

- (1) *Complete History, W/O Skyline*, feeds the full raw conversation history of the LLM at every iteration and disables the skyline component concatenated to the AssignmentLM's prompt.
- (2) *Summarized History, W/O Skyline*, utilizes history-summarization instead of the full conversation log, with skyline tracking by the AssignmentLM disabled.

| Model                 | Setting                | Success Rate (%) |              |              |              | Refinement Optimality (%) |              |              |              |
|-----------------------|------------------------|------------------|--------------|--------------|--------------|---------------------------|--------------|--------------|--------------|
|                       |                        | Top-K            | Range        | Diversity    | Complex      | Top-K                     | Range        | Diversity    | Complex      |
| GPT-4.1-mini          | No-SubspaceLM          | 70.0             | 75.0         | 75.0         | 65.0         | 72.4                      | 51.6         | 73.7         | 63.2         |
|                       | No-AssignmentLM        | 82.5             | 22.5         | 57.5         | 62.5         | 83.71                     | 21.20        | 72.79        | 68.73        |
|                       | <b>OmniTune (Full)</b> | <b>97.5</b>      | <b>97.5</b>  | <b>90.0</b>  | <b>98.0</b>  | <b>96.0</b>               | <b>85.5</b>  | <b>96.5</b>  | <b>93.8</b>  |
| Gemini-2.0-Flash-Lite | No-SubspaceLM          | 90.0             | 90.5         | 47.5         | 50.0         | 90.73                     | 66.81        | 57.40        | 76.37        |
|                       | No-AssignmentLM        | 95.0             | 92.5         | 57.5         | 72.5         | 91.52                     | 68.42        | 77.24        | 80.16        |
|                       | <b>OmniTune (Full)</b> | <b>95.0</b>      | <b>100.0</b> | <b>75.0</b>  | <b>87.5</b>  | <b>94.75</b>              | <b>76.71</b> | <b>92.50</b> | <b>92.34</b> |
| Mistral-Small-3.1-24B | No-SubspaceLM          | 62.5             | 87.5         | 37.5         | 100.0        | 48.63                     | 63.07        | 34.59        | 85.55        |
|                       | No-AssignmentLM        | 100.0            | 100.0        | 75.0         | 87.5         | 77.50                     | 63.10        | 67.74        | 63.29        |
|                       | <b>OmniTune (Full)</b> | <b>100.0</b>     | <b>100.0</b> | <b>100.0</b> | <b>100.0</b> | <b>91.06</b>              | <b>87.43</b> | <b>95.27</b> | <b>85.78</b> |

Table 4: Two-step OPRO scheme ablation Study: Success Rate and Refinement Optimality.

| Model                 | Setting                                 | Success Rate (%) |              |              |              | Refinement Optimality (%) |              |              |              |
|-----------------------|-----------------------------------------|------------------|--------------|--------------|--------------|---------------------------|--------------|--------------|--------------|
|                       |                                         | Top-K            | Range        | Diversity    | Complex      | Top-K                     | Range        | Diversity    | Complex      |
| GPT-4.1-mini          | Complete History (W/O Skyline)          | 70.0             | 25.0         | 37.5         | 42.5         | 59.88                     | 20.68        | 63.95        | 44.85        |
|                       | Summarized History (W/O Skyline)        | 90.0             | 92.5         | 67.5         | 87.5         | 94.49                     | 69.62        | 92.23        | 77.29        |
|                       | <b>OmniTune (Summ. History+Skyline)</b> | <b>97.5</b>      | <b>97.5</b>  | <b>90.0</b>  | <b>98.0</b>  | <b>96.0</b>               | <b>85.5</b>  | <b>96.5</b>  | <b>93.8</b>  |
| Gemini-2.0-Flash-Lite | Complete History (W/O Skyline)          | 22.5             | 42.5         | 12.5         | 17.5         | 12.51                     | 23.18        | 11.55        | 39.60        |
|                       | Summarized History (W/O Skyline)        | 90.0             | 87.5         | 52.5         | 87.5         | 94.07                     | 64.44        | 82.54        | 75.01        |
|                       | <b>OmniTune (Summ. History+Skyline)</b> | <b>95.0</b>      | <b>100.0</b> | <b>75.0</b>  | <b>87.5</b>  | <b>94.75</b>              | <b>76.71</b> | <b>92.50</b> | <b>92.34</b> |
| Mistral-Small-3.1-24B | Complete History (W/O Skyline)          | 62.5             | 87.5         | 75.0         | 75.0         | 33.51                     | 60.79        | 67.98        | 64.35        |
|                       | Summarized History (W/O Skyline)        | 100.0            | 75.0         | 62.5         | 87.5         | 91.00                     | 49.67        | 57.49        | 69.26        |
|                       | <b>OmniTune (Summ. History+Skyline)</b> | <b>100.0</b>     | <b>100.0</b> | <b>100.0</b> | <b>100.0</b> | <b>91.06</b>              | <b>87.43</b> | <b>95.27</b> | <b>85.78</b> |

Table 5: History Structures Ablation Study: Success Rate and Refinement Optimality.

Table 5 presents the resulting success rates and optimality values. Across all models and refinement classes OMNITUNE, when equipped with both components (history summarization and the skyline), achieves the highest success rates and optimality values, demonstrating the value of these components within the system. We observed a significant improvement of an average of 48.3% in success rate when using OMNITUNE compared to the complete history and without the skyline component, with a prominent improvement of 86.8% for the Diversity class when using Gemini. While using summarized history (without the skyline component) shows improvement over full history in most cases (with the exception of some classes when using the Mistral model), we observed an average improvement of 37.3% when using both, summarized history and the skyline component in OMNITUNE.

#### 5.4 Effect of Parameters

Next, we analyze the impact of the horizon  $T$  and per-subspace refinements  $K$  on OMNITUNE, measuring success rate, optimality, and token cost as  $T, K \in [1, 10]$  using GPT-4o-mini.

*Refinements per Iteration ( $K$ ).* In Figures 8, 9 and 10 we report the success rates, optimality values, and token cost, respectively, when ranging  $K$  from 1 to 10 using different (fixed) number of subspaces ( $T$ ). As expected, we observed an overall improvement in most cases in the success rates and optimality values with the cost of a growing amount of tokens sent to the LLM model as  $K$

increases. For the Top- $k$ , Range and Diversity instances (Figures 8a-8c and 9a-9c), when the horizon is set to 5 or more, the incremental gains in both success rates and optimality become negligible. For Complex instances (Figures 8d and 9d), the resulting success and optimality did not show a clear trend when using a low number of subspaces (i.e., 1-3 subspaces). This is due to the complex structure of the queries, e.g., including a HAVING clause predicates, which requires a larger number of iterations to improve the quality of the output. The observed token cost (Figure 10) showed, as expected, consistent, linear growth in tokens used as  $K$  increases.

*Horizon ( $T$ ).* Figures 11, 12, and 13 show the effect of varying the horizon  $T$  for different (fixed) numbers of refinements per iteration. Consistent with the impact of increasing refinements per iteration, increasing  $T$ , as expected, the overall trend is growth in both the quality of the solution (success rate and optimality values) as well as the token cost. We observed some deviations with respect to this trend in the quality of the results for the Top- $k$  class, likely due to the inherent complexity of the constraint objectives in this class, which can involve internal trade-offs in certain scenarios. As with  $K$ , we observed a diminishing improvement in the success rate and optimality beyond the point of  $T = 5$  when using 5 or more refinements per iteration. In contrast to the linear growth observed in the token cost when increasing  $K$ , we observed only a moderate growth for increasing values of  $T$  (Figure 13). This is due to the effect of a higher number of refinements per subspace, leading to faster

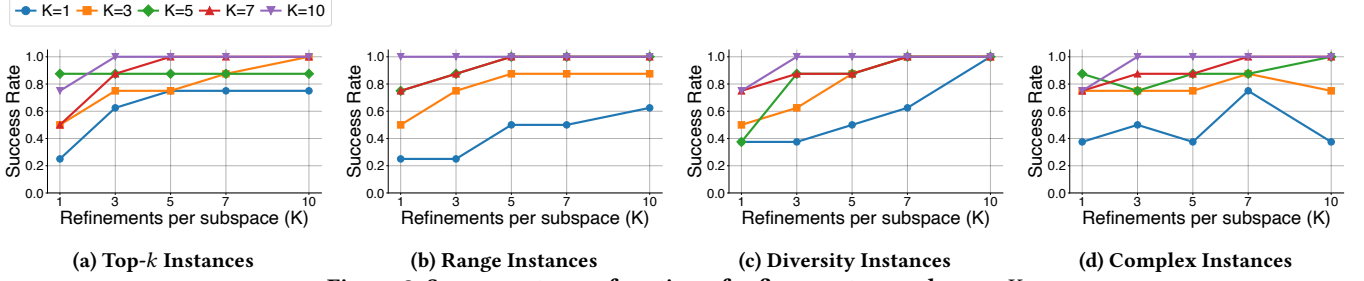


Figure 8: Success rate as a function of refinements per subspace  $K$

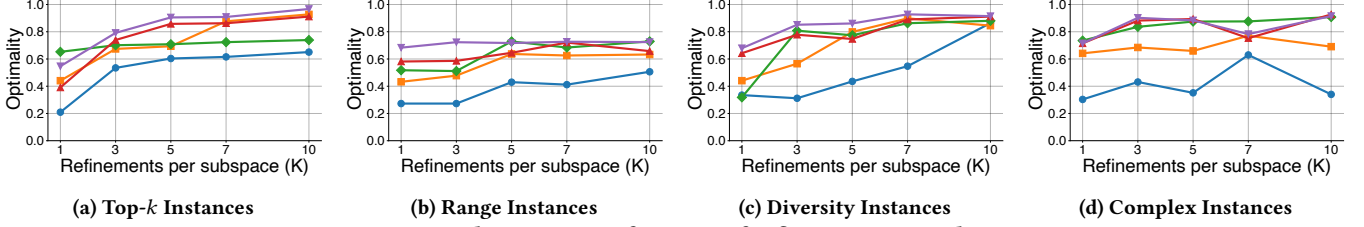


Figure 9: Optimality score as a function of refinements per subspace  $K$ .

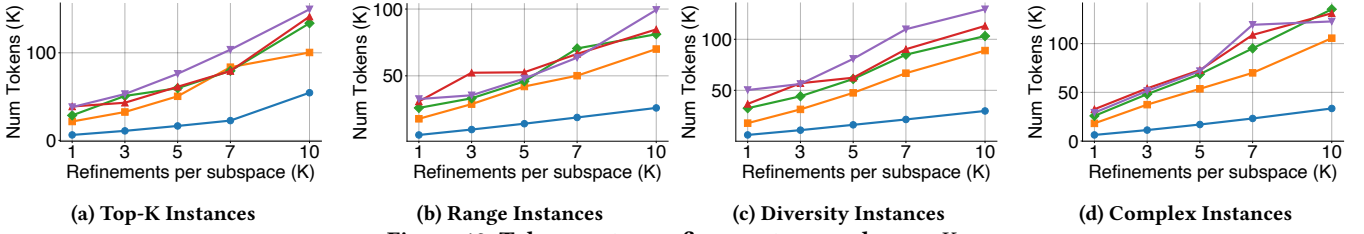


Figure 10: Token-cost vs. refinements per subspace  $K$ .

convergence in finding the minimal refinement. Thus, coupled with the early stopping mechanism, the system requires fewer subspace iterations to yield a successful refinement and terminate.

## 6 Related Work

**Query refinement.** The problem of query refinement, i.e., modifying queries to satisfy some output constraints, was studied in a line of work [17, 18, 23, 37, 42, 43, 46, 48, 49, 54, 57, 58]. For example, [37, 49] aim to relax queries with an empty result set to produce some answers. Other works like [23, 46] address the issues of too many or too few answers by refining queries to meet specific cardinality constraints on the result’s size. A recent line of work has studied the use of refinement to satisfy diversity constraints [17, 18, 42, 43, 54] and primarily focused on SPJ queries, where a refinement of a query is a set of modifications to the constants in the WHERE clause. The authors of [54] consider range queries, namely, selection queries with conjunction of range conditions over predicates of the data, and [17, 18] proposed an automated approach using linear optimization techniques to refine SPJ queries with an ORDER BY clause. Different notions of similarity measures used to define the minimality of refinements were defined in these works. E.g., in [54], the similarity between queries is measured by the Jaccard similarity of their outputs, whereas in [42, 43], the similarity between queries is defined with respect to the values of the constants in their WHERE clause. Our proposed solution

generalizes the concept of query refinements to support a broader class of queries, constraints, and similarity measures. Imposing constraints over answer sets and performing query relaxation has also been studied in the context of package queries [15, 16]. However, because package queries are not standard SQL, supporting them was outside the scope of this paper.

**LLMs for SQL Generation.** Recent work has demonstrated that large language models (LLMs) achieve state-of-the-art performance on the text-to-SQL task [28, 65], in which users express their information needs in natural language and the system generates a SQL query whose execution answers those needs [35, 67]. LLM-based solutions surpass earlier dedicated architectures [30, 64], primarily due to their strong semantic reasoning capabilities, which enable them to jointly interpret underspecified user intent and database schemas. Accordingly, most text-to-SQL systems are designed around single-pass query synthesis pipelines, commonly emphasizing schema linking [27, 41, 61], clause-level SQL generation [38], and execution-based validation mechanisms [63] that are primarily used to detect and correct syntactic errors or mismatches with the database schema.

In contrast, our setting considers a substantially different problem. Rather than generating a SQL query from a natural language description, the model is given an existing query and is tasked with refining it so as to satisfy a set of constraints while minimizing deviation from the original query. This refinement process inherently

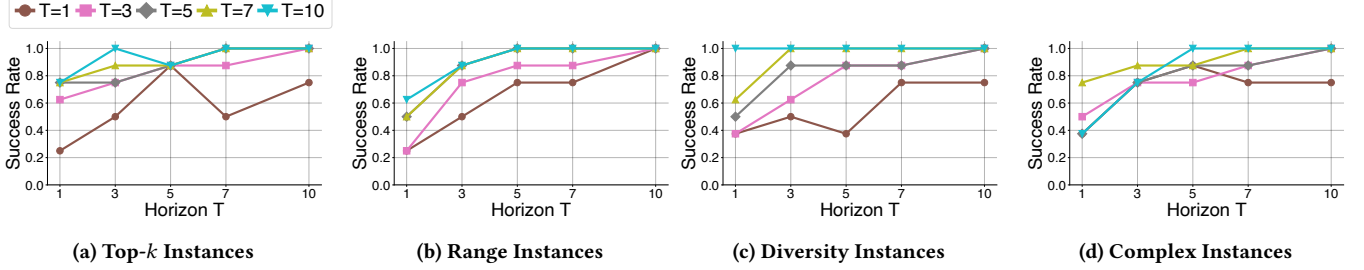


Figure 11: Success rate as a function of the number of subspaces  $T$ .

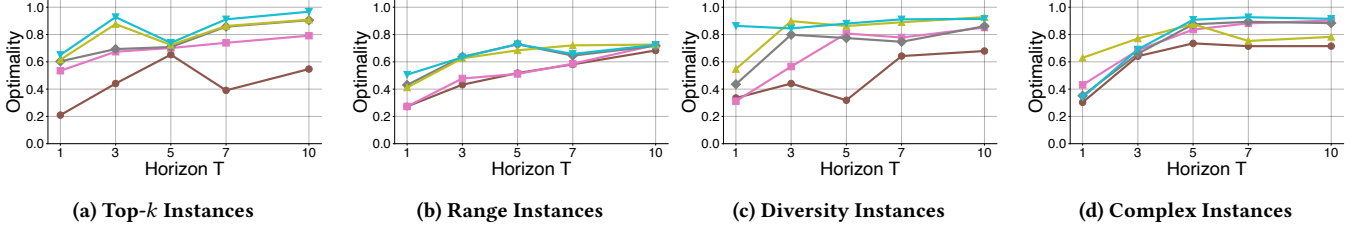


Figure 12: Optimality score as a function of the number of subspaces  $T$ .

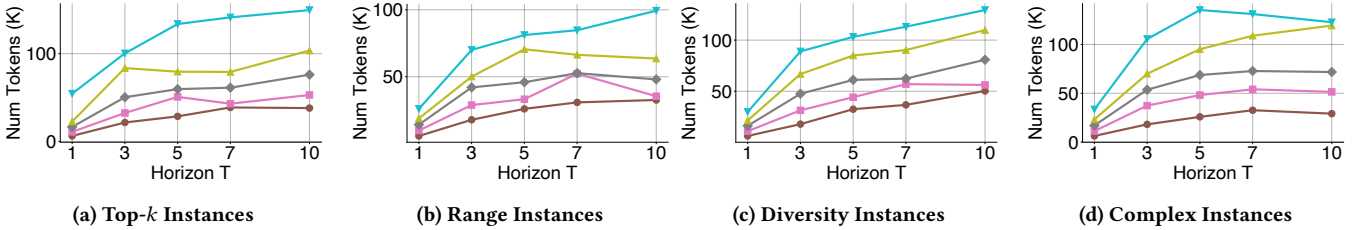


Figure 13: Token-cost vs. number of subspaces (Horizon  $T$ ).

requires iterative reasoning over query modifications, explicit trade-offs between competing objectives, and feedback from previous attempts, which goes beyond execution-based correctness checks and fundamentally differs from the one-shot generation paradigm underlying most text-to-SQL approaches.

**Black-Box and LLM-Based Optimization.** In contrast to *white-box* optimization, where the objective function and constraints are explicitly specified and structurally accessible to the solver, *black-box* optimization treats the objective function as an oracle that can only be evaluated at sampled points, without access to its internal structure. Representative approaches include Monte-Carlo sampling, multi-armed bandits, evolutionary algorithms, and related population-based methods [25, 32, 34, 36, 60]. While broadly applicable, these methods rely on the ability to meaningfully sample the search space and learn solely from observed objective values, often requiring extensive exploration and struggling in large, discrete, or highly discontinuous domains.

Recently, large language models have been proposed as optimization engines in settings where both the search space and the objective can be expressed in natural language. Prior work shows that LLMs can optimize prompts, heuristics, and symbolic programs—through iterative prompting [19, 22, 39, 45, 62]. However, existing OPRO-style approaches degrade in large, discrete, and

combinatorial domains, where naïve candidate generation and unstructured interaction histories provide weak guidance and limit scalability [62, 66].

To address these limitations, we build on the OPRO framework and introduce a dedicated two-step scheme for query refinement, explicitly structuring both the refinement search space and the optimization feedback to guide LLMs more effectively.

## 7 Conclusion

We presented OMNITUNE, a general framework for SQL query refinement that seeks refinements minimizing distance to an original query while approximately satisfying user-defined constraints. Our approach adapts the OPRO paradigm to query refinement via a dedicated two-step scheme that combines structured subspace exploration with focused refinement sampling, guided by compact history summarization and skyline-based feedback. An extensive empirical evaluation demonstrates that OMNITUNE is effective and robust across a broad range of existing and novel query refinement scenarios beyond the scope of prior solutions.

In future work, we plan to further enhance the OPRO scheme and the OMNITUNE design, for example by incorporating skyline-aware exploration-exploitation strategies and extending the LLM modules into agentic components capable of issuing and analyzing auxiliary analytical queries.

## References

- [1] 2016. Medical Expenditure Panel Survey Dataset. [https://meps.ahrq.gov/data\\_stats/download\\_data/pufs/h192/h192doc.shtml](https://meps.ahrq.gov/data_stats/download_data/pufs/h192/h192doc.shtml). (2016).
- [2] 2016. NASA Astronauts, 1959-Present (Kaggle). <https://www.kaggle.com/datasets/nasa/astronaut-yearbook>. (2016).
- [3] 2017. COMPAS Dataset (Kaggle). <https://www.kaggle.com/datasets/danofer/compass>. (2017).
- [4] 2018. ACS Income Dataset. <https://github.com/socialfoundations/folktables>. (2018).
- [5] 2018. Students Performance in Exams (Kaggle). <https://www.kaggle.com/datasets/spscientist/students-performance-in-exams>. (2018).
- [6] 2019. TPC-H Dataset. <https://github.com/aleaugustoplus/tpch-data>. (2019).
- [7] 2021. Healthcare Dataset. [https://github.com/stefan-grafberger/mlinspect/tree/master/example\\_pipelines/healthcare](https://github.com/stefan-grafberger/mlinspect/tree/master/example_pipelines/healthcare). (2021).
- [8] 2021. Housing Prices Dataset (Kaggle). <https://www.kaggle.com/datasets/yasserh/housing-prices-dataset>. (2021).
- [9] 2022. Law School Admissions Bar Passage Dataset (Kaggle). <https://www.kaggle.com/datasets/danofer/law-school-admissions-bar-passage>. (2022).
- [10] 2022. Texas Tribune Dataset (Kaggle). <https://www.kaggle.com/datasets/mexwell/texas-tribune-government-salaries/data>. (2022).
- [11] 2023. LangChain Python Library. <https://github.com/langchain-ai/langchain>. (2023).
- [12] 2024. Fraud Detection Transactions Dataset (Kaggle). <https://www.kaggle.com/datasets/samayashar/fraud-detection-transactions-dataset>. (2024).
- [13] 2024. Pydantic Validation Python Library. <https://github.com/pydantic/pydantic>. (2024).
- [14] S. Borzsony, Donald Kossmann, and Konrad Stocker. 2001. The Skyline Operator. *Proceedings - International Conference on Data Engineering*, 421 – 430. <https://doi.org/10.1109/ICDE.2001.914855>
- [15] Matteo Brucato, Azza Abouzied, and Alexandra Meliou. 2014. Improving package recommendations through query relaxation. In *Proceedings of the First International Workshop on Bringing the Value of "Big Data" to Users (Data4U 2014)*. 13–18.
- [16] Matteo Brucato, Juan Felipe Beltran, Azza Abouzied, and Alexandra Meliou. 2016. Scalable Package Queries in Relational Database Systems. *Proc. VLDB Endow.* 9, 7 (2016), 576–587. <https://doi.org/10.14778/2904483.2904489>
- [17] Felix S. Campbell, Alon Silberstein, Julia Stoyanovich, and Yuval Moskovitch. 2024. Query Refinement for Diverse Top-k Selection. *Proc. ACM Manag. Data* 2, 3 (2024), 166. <https://doi.org/10.1145/3654969>
- [18] Felix S. Campbell, Julia Stoyanovich, and Yuval Moskovitch. 2024. Rodeo: Making Refinements for Diverse Top-k Queries. *Proc. VLDB Endow.* 17, 12 (2024), 4341–4344. <https://doi.org/10.14778/3685800.3685870>
- [19] Linnan Chen, Hao Tan, Tianyu Gao, Logan Dery, Ruilian Jiang, Xiyu Liu, Ce Deng, and Feng Li. 2022. Multi-step Planning for Automated Hyperparameter Optimization with OptFormer. *arXiv preprint arXiv:2210.04971* (2022).
- [20] Wenhui Chen, Xueguang Ma, Xinyi Wang, and William W. Cohen. 2023. Program of Thoughts Prompting: Disentangling Computation from Reasoning for Numerical Reasoning Tasks. *Transactions on Machine Learning Research* (2023). <https://openreview.net/forum?id=YfZ4ZPt8zd>
- [21] Yongchao Chen, Jacob Arkin, Yilun Hao, Yang Zhang, Nicholas Roy, and Chuchu Fan. 2024. PROMPT Optimization in Multi-Step Tasks (PROMST): Integrating Human Feedback and Heuristic-based Sampling. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (Eds.). Association for Computational Linguistics, Miami, Florida, USA, 3859–3920. <https://doi.org/10.18653/v1/2024.emnlp-main.226>
- [22] Zhiyu Chen, Qiusi Guo, Zifan Cai, Xiang Zhang, and Yujiu Wang. 2023. Evo-Prompt: Connecting LLMs with Evolutionary Algorithms Yields Powerful Prompt Optimizers. *arXiv preprint arXiv:2309.08532* (2023).
- [23] Wesley W. Chu and Qiming Chen. 1994. A Structured Approach for Cooperative Query Answering. *IEEE Trans. Knowl. Data Eng.* 6, 5 (1994), 738–749. <https://doi.org/10.1109/69.317704>
- [24] Gheorghe Comanici, Eric Bieber, Mike Schaekermann, Ice Pasupat, Noveen Sachdeva, Inderjit Dhillon, and more. 2025. *Gemini 2.5: Pushing the Frontier with Advanced Reasoning, Multimodality, Long Context, and Next Generation Agentic Capabilities*. Technical Report. [arXiv:cs.CL/2507.06261](https://arxiv.org/abs/2507.06261) <https://arxiv.org/abs/2507.06261>
- [25] Remi Coulom. 2006. Efficient Selection and Backup Operators in Monte-Carlo Tree Search. In *Computers and Games, 5th International Conference, CG 2006, Turin, Italy, May 22-24, 2006, Revised Selected Papers (Lecture Notes in Computer Science)*, Vol. 4630. Springer, 72–83. [https://doi.org/10.1007/978-3-540-75540-8\\_7](https://doi.org/10.1007/978-3-540-75540-8_7)
- [26] DuckDB. n.d. *DuckDB: Documentation*. <https://duckdb.org/docs/stable/> Online documentation.
- [27] Authors from ACL Anthology. 2025. GenLink: Generation-Driven Schema-Linking via Multi-Task Fine-Tuning for Text-to-SQL. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 1518. <https://aclanthology.org/2025.emnlp-main.1518/>
- [28] Dawei Gao, Haibin Wang, Yaliang Li, Xiuyu Sun, Yichen Qian, Bolin Ding, and Jingren Zhou. 2024. Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation. *Proc. VLDB Endow.* 17, 5, 1132–1145. <https://doi.org/10.14778/3641204.3641221>
- [29] OmniTune Github. 2024. <https://github.com/analysis-bots/OmniTune>. (2024).
- [30] Jiaqi Guo, Zecheng Zhan, Yan Gao, Yan Xiao, Jian-Guang Lou, Ting Liu, and Dongmei Zhang. 2019. Towards Complex Text-to-SQL in Cross-Domain Database with Intermediate Representation. In *Proceedings of the 57th Conference of the Association for Computational Linguistics, ACL 2019, Florence, Italy, July 28–August 2, 2019, Volume 1: Long Papers*, Anna Korhonen, David R. Traum, and Lluís Màrquez (Eds.). Association for Computational Linguistics, 4524–4535. <https://doi.org/10.18653/V1/P19-1444>
- [31] Eldar Hachohen, Yuval Moskovitch, and Amit Somech. 2025. OmniTune: A Universal Framework for Query Refinement via LLMs. In *Companion of the 2025 International Conference on Management of Data, SIGMOD/PODS 2025, Berlin, Germany, June 22–27, 2025*, Volker Markl, Joseph M. Hellerstein, and Azza Abouzied (Eds.). ACM, 111–114. <https://doi.org/10.1145/3722212.3725121>
- [32] Nikolaus Hansen and Andreas Ostermeier. 2001. Completely Derandomized Self-Adaptation in Evolution Strategies. *Evolutionary Computation* 9, 2 (2001), 159–195.
- [33] Jie Jiang, Haining Xie, Siqi Shen, Yu Shen, Zihan Zhang, Meng Lei, Yifeng Zheng, Yang Li, Chunyou Li, Danqing Huang, Yinjun Wu, Wentao Zhang, Xiaofeng Yang, Bin Cui, and Peng Chen. 2025. SiriusBI: A Comprehensive LLM-powered Solution for Data Analytics in Business Intelligence. *Proc. VLDB Endow.* 18, 12 (2025), 4860–4873. <https://www.vldb.org/pvldb/vol18/p4860-xie.pdf>
- [34] James Kennedy and Russell Eberhart. 1995. Particle Swarm Optimization. In *Proceedings of the IEEE International Conference on Neural Networks*. 1942–1948.
- [35] Hyeonji Kim, Byeong-Hoon So, Wook-Shin Han, and Hongrae Lee. 2020. Natural language to SQL: where are we today? *Proc. VLDB Endow.* 13, 10 (June 2020), 1737–1750. <https://doi.org/10.14778/3401960.3401970>
- [36] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi. 1983. Optimization by Simulated Annealing. *Science* 220, 4598 (1983), 671–680.
- [37] Nick Koudas, Chen Li, Anthony K. H. Tung, and Rares Vernica. 2006. Relaxing Join and Selection Queries. In *Proceedings of the 32nd International Conference on Very Large Data Bases, Seoul, Korea, September 12–15, 2006*, Umeshwar Dayal, Kyu-Young Whang, David B. Lomet, Gustavo Alonso, Guy M. Lohman, Martin L. Kersten, Sang Kyun Cha, and Young-Kuk Kim (Eds.). ACM, 199–210. <http://dl.acm.org/citation.cfm?id=1164146>
- [38] Dongjun Lee. 2019. Clause-Wise and Recursive Decoding for Complex and Cross-Domain Text-to-SQL Generation. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, Kentaro Inui, Jing Jiang, Vincent Ng, and Xiaojun Wan (Eds.). Association for Computational Linguistics, Hong Kong, China, 6045–6051. <https://doi.org/10.18653/v1/D19-1624>
- [39] Joel Lehman, Jonathan Gordon, Shawn Jain, Kamal Ndousse, Cathy Yeh, and Kenneth O. Stanley. 2022. Evolution through Large Models. *arXiv preprint arXiv:2206.08896* (2022).
- [40] Mosh Levy, Alon Jacoby, and Yoav Goldberg. 2024. Same Task, More Tokens: the Impact of Input Length on the Reasoning Performance of Large Language Models. In *62nd Annual Meeting of the Association for Computational Linguistics, ACL 2024*. Association for Computational Linguistics (ACL), 15339–15353.
- [41] Haoyang Li, Jing Zhang, Cuiping Li, and Hong Chen. 2023. RESDSL: decoupling schema linking and skeleton parsing for text-to-SQL. In *Proceedings of the Thirty-Seventh AAAI Conference on Artificial Intelligence and Thirtieth Symposium on Innovative Applications of Artificial Intelligence and Thirteenth Symposium on Educational Advances in Artificial Intelligence (AAAI’23/IAAI’23/EAAT’23)*. AAAI Press, Article 1466, 9 pages. <https://doi.org/10.1609/aaai.v37i1.126535>
- [42] Jinyang Li, Yuval Moskovitch, Julia Stoyanovich, and H. V. Jagadish. 2023. Query Refinement for Diversity Constraint Satisfaction. *Proc. VLDB Endow.* 17, 2 (2023), 106–118. <https://doi.org/10.14778/3626292.3626295>
- [43] Jinyang Li, Alon Silberstein, Yuval Moskovitch, Julia Stoyanovich, and H. V. Jagadish. 2023. ERICA: Query Refinement for Diversity Constraint Satisfaction. *Proc. VLDB Endow.* 16, 12 (2023), 4070–4073. <https://doi.org/10.14778/3611540.3611623>
- [44] Yurong Liu, Eduardo H. M. Pena, Aécio Santos, Eden Wu, and Juliana Freire. 2025. Magneto: Combining Small and Large Language Models for Schema Matching. *Proc. VLDB Endow.* 18, 8 (April 2025), 2681–2694. <https://doi.org/10.14778/3742728.3742757>
- [45] Eric Meyerson, Jason Nelson, Matthew Bradley, Adam Gaier, Hanie Moradi, Alexander Hoover, and Joel Lehman. 2023. Language Model Crossover: Variation through Few-Shot Prompting. *arXiv preprint arXiv:2307.12043* (2023).
- [46] Chaitanya Mishra and Nick Koudas. 2009. Interactive query refinement. In *EDBT 2009, 12th International Conference on Extending Database Technology, Saint Petersburg, Russia, March 24–26, 2009, Proceedings (ACM International Conference Proceeding Series)*, Martin L. Kersten, Boris Novikov, Jens Teubner, Vladimir Polutin, and Stefan Manegold (Eds.), Vol. 360. ACM, 862–873. <https://doi.org/10.1145/1516360.1516459>

- [47] Mistral-AI, :, Abhinav Rastogi, Albert Q. Jiang, Andy Lo, Gabrielle Berrada, Guillaume Lample, and more. 2025. *Magistral*. Technical Report. arXiv:cs.CL/2506.10910 <https://arxiv.org/abs/2506.10910>
- [48] Yuval Moskovitch, Jinyang Li, and H. V. Jagadish. 2022. Bias analysis and mitigation in data-driven tools using provenance. In *Proceedings of the 14th International Workshop on the Theory and Practice of Provenance, TaPP 2022, Philadelphia, Pennsylvania, 17 June 2022*, Adriane Chapman, Daniel Deutch, and Tanu Malik (Eds.). ACM, 1:1–1:4. <https://doi.org/10.1145/3530800.3534528>
- [49] Ion Muslea and Thomas J. Lee. 2005. Online Query Relaxation via Bayesian Causal Structures Discovery. In *Proceedings, The Twentieth National Conference on Artificial Intelligence and the Seventeenth Innovative Applications of Artificial Intelligence Conference, July 9-13, 2005, Pittsburgh, Pennsylvania, USA*, Manuela M. Veloso and Subbarao Kambhampati (Eds.). AAAI Press / The MIT Press, 831–836. <http://www.aaai.org/Library/AAAI/2005/aaai05-131.php>
- [50] OpenAI. 2025. *GPT-5 System Card*. Technical Report. OpenAI. <https://cdn.openai.com/gpt-5-system-card.pdf> System card.
- [51] OpenAI, Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, and more. 2024. *GPT-4 Technical Report*. Technical Report. arXiv:cs.CL/2303.08774 <https://arxiv.org/abs/2303.08774>
- [52] Lutz Prechelt. 2002. Early stopping-but when? In *Neural Networks: Tricks of the trade*. Springer, 55–69.
- [53] Alex A. T. Rathke. 2025. *A representation theorem for events within lattice structures of state-spaces*. Papers 2505.18615. arXiv.org. <https://doi.org/None>
- [54] Suraj Shetiya, Ian P. Swift, Abolfazl Asudeh, and Gautam Das. 2022. Fairness-Aware Range Queries for Selecting Unbiased Data. In *38th IEEE International Conference on Data Engineering, ICDE 2022, Kuala Lumpur, Malaysia, May 9-12, 2022*. IEEE, 1423–1436. <https://doi.org/10.1109/ICDE53745.2022.00111>
- [55] Sonish Sivarajkumar, Mark Kelley, Alyssa Samolyk-Mazzanti, Shyam Visweswaran, and Yanshan Wang. 2024. An Empirical Evaluation of Prompting Strategies for Large Language Models in Zero-Shot Clinical Natural Language Processing: Algorithm Development and Validation Study. *JMIR Med Inform* 12 (Apr 2024), e55318. <https://doi.org/10.2196/55318>
- [56] Deshan Koshala Sumanathilaka, Nicholas Micallef, and Julian Hough. 2024. Can LLMs assist with Ambiguity? A Quantitative Evaluation of various Large Language Models on Word Sense Disambiguation. *Proceedings of the First International Conference on Natural Language Processing and Artificial Intelligence for Cyber Security* (2024). <https://aclanthology.org/2024.nlpaics-1.12/>
- [57] Quoc Trung Tran and Chee-Yong Chan. 2010. How to ConQueR why-not questions. In *Proceedings of the ACM SIGMOD International Conference on Management of Data, SIGMOD 2010, Indianapolis, Indiana, USA, June 6-10, 2010*, Ahmed K. Elmagarmid and Divyakant Agrawal (Eds.). ACM, 15–26. <https://doi.org/10.1145/1807167.1807172>
- [58] Quoc Trung Tran, Chee-Yong Chan, and Srinivasan Parthasarathy. 2009. Query by output. In *Proceedings of the ACM SIGMOD International Conference on Management of Data, SIGMOD 2009, Providence, Rhode Island, USA, June 29 - July 2, 2009*, Ugur Çetintemel, Stanley B. Zdonik, Donald Kossmann, and Nesime Tatbul (Eds.). ACM, 535–548. <https://doi.org/10.1145/1559845.1559902>
- [59] Immanuel Trummer. 2024. Generating Succinct Descriptions of Database Schemata for Cost-Efficient Prompting of Large Language Models. *Proceedings of the VLDB Endowment* 17, 11 (2024), 3511–3523. <https://www.vldb.org/pvldb/vol17/p3511-trummer.pdf>
- [60] Niki van Reijß. 2018. *Monte Carlo Tree Search for Combinatorial Optimization*. Ph.D. Dissertation. Radboud University. <https://dblp.org/rec/phd/pt/Rei18.html>
- [61] Yihan Wang, Peiyu Liu, and Xin Yang. 2025. LinkAlign: Scalable Schema Linking for Real-World Large-Scale Multi-Database Text-to-SQL. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. <https://aclanthology.org/2025.emnlp-main.51>
- [62] Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. 2024. Large Language Models as Optimizers. (2024). arXiv:cs.LG/2309.03409 <https://arxiv.org/abs/2309.03409>
- [63] Yicun Yang, Zhaoguo Wang, Yu Xia, Zhuoran Wei, Haoran Ding, Ruzica Piskac, Haibo Chen, and Jinyang Li. 2025. Automated Validating and Fixing of Text-to-SQL Translation with Execution Consistency. *Proc. ACM Manag. Data* 3, 3, Article 134 (June 2025), 28 pages. <https://doi.org/10.1145/3725271>
- [64] Tao Yu, Michihiro Yasunaga, Kai Yang, Rui Zhang, Dongxu Wang, Zifan Li, and Dragomir Radev. 2018. SyntaxSQLNet: Syntax Tree Networks for Complex and Cross-Domain Text-to-SQL Task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, Ellen Riloff, David Chiang, Julia Hockenmaier, and Jun'ichi Tsujii (Eds.). Association for Computational Linguistics, Brussels, Belgium, 1653–1663. <https://doi.org/10.18653/v1/D18-1193>
- [65] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. 2018. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, Ellen Riloff, David Chiang, Julia Hockenmaier, and Jun'ichi Tsujii (Eds.). Association for Computational Linguistics, Brussels, Belgium, 3911–3921. <https://doi.org/10.18653/v1/D18-1425>
- [66] Tuo Zhang, Jinyue Yuan, and Salman Avestimehr. 2024. Revisiting OPRO: The Limitations of Small-Scale LLMs as Optimizers. In *Findings of the Association for Computational Linguistics: ACL 2024*. Association for Computational Linguistics, Bangkok, Thailand, 1727–1735. <https://aclanthology.org/2024.findings-acl.100>
- [67] Yuze Zhang et al. 2025. EVOSHEMA: Towards Text-to-SQL Robustness against Schema Evolution. *Proc. VLDB Endow.* 18 (2025). <https://www.vldb.org/pvldb/vol18/p3655-zhang.pdf>